ANA RAQUEL FERNANDES CUNHA

PDS TECH

IA PARA DEVS

IAs GENERATIVAS

AULA 03

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

SUMÁRIO

O QUE VEM POR AÍ?.....3
HANDS ON 4
SAIBA MAIS.....5
O QUE VOCÊ VIU NESTA AULA? 27
REFERÊNCIAS.....28

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

O QUE VEM POR AÍ?

Você sabia que as IAs generativas podem ser utilizadas para a geração de códigos, imagens e até vídeos? Pois é, incrível não é mesmo? Imagine as possibilidades para a geração de conteúdo e a produtividade que pode gerar em nosso dia a dia?! Nesta aula, você vai aprender como é o funcionamento dessas tecnologias e como podemos utilizá-las de forma produtiva.

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

HANDS ON

Nas videoaulas você aprenderá como as IA generativas funcionam por trás da “mágica” para criar imagens, textos e vídeos. Você vai aprender tanto a teoria, como a prática desses modelos geradores de conteúdo.

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

SAIBA MAIS

Entendendo a criação de imagens com a IA generativa

Na aula de introdução você conheceu um pouquinho de como é o funcionamento das IAs generativas para a criação de imagens, as famosas **GANs** (Redes Adversárias Generativas), também podendo ser encontradas nas literaturas da área com o nome "**Difusão"**. O nome difusão não existe à toa, essa rede consegue criar imagens novas com a "disputa" entre duas redes: uma **rede geradora** responsável por tentar gerar novos dados (novas imagens) e uma **rede discriminadora** que tentará distinguir imagens reais. Falando de uma forma simples, a rede geradora faz o papel de "iludida", tentando enganar a rede discriminadora de que as imagens criadas por ela são realmente reais e não fakes (brincadeiras à parte!).

Essa rede funciona da seguinte maneira: a rede geradora considera imagens aleatórias e retorna uma delas. Em seguida, essa imagem é selecionada de forma aleatória e é inserida na rede discriminadora ao lado de um fluxo de imagens reais. A rede discriminadora obtém as imagens reais e as falsas, e retorna a probabilidade da imagem ser autêntica, ou seja, ser real.

Esse loop de feedback entre as redes ocorre dentro de uma rede neural convolucional padrão, que categoriza as imagens como reais e fakes, enquanto a rede geradora é uma rede convolucional inversa, adicionando um ruído aleatório na imagem, realizando uma técnica chamada **upsampling** (também conhecido como superamostragem, que é a técnica de **aumentar a resolução espacial** de uma imagem. Em CNNs, essa técnica é frequentemente utilizada após o processo de convolução e pooling, que tende a reduzir as dimensões da imagem).

Essa etapa existe justamente para tentar criar novas imagens pela rede geradora, criando pixels adicionais e refinando os detalhes da imagem, aproximando-a cada vez mais da realidade. A camada da discriminadora é uma camada convolucional que realiza a técnica de **downsampling**, utilizada para **reduzir a dimensionalidade** das imagens reais e geradas, facilitando o processo de análise e distinção entre elas. Técnicas como **Max Pooling** ou **Average Pooling** diminuem o tamanho das imagens sem perder informações cruciais para a tomada de decisão.

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

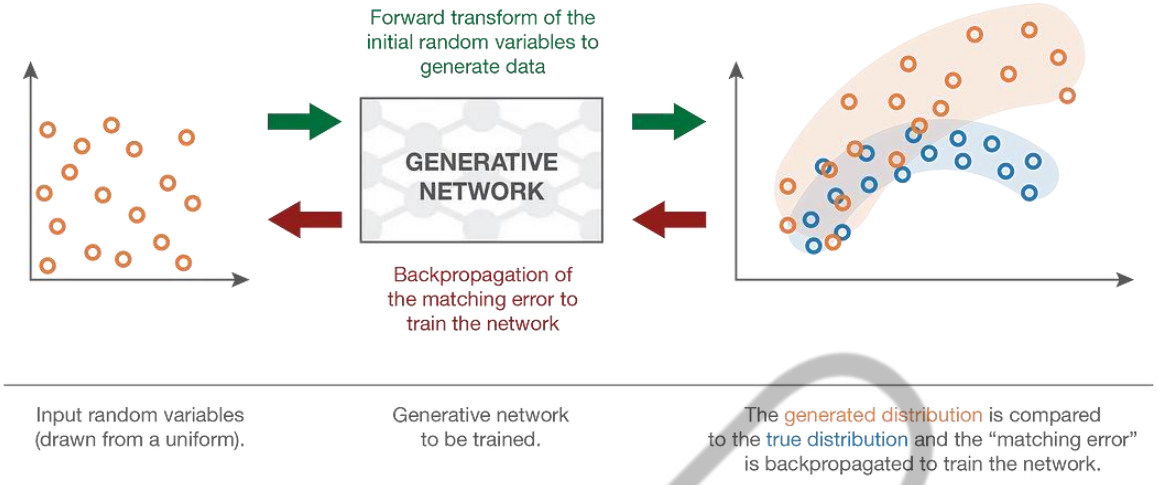


Figura 1 – Redes de correspondência generativa
Fonte: (https://towardsdatascience.com/understanding-generative-adversarial-networks-gans-cd6e4651a29 (2019)

As redes de correspondência generativa recebem entradas aleatórias simples, geram novos dados, comparam diretamente a distribuição dos dados gerados com a distribuição dos dados verdadeiros e propagam retroativamente o erro de correspondência para treinar a rede.

Ambas as redes tentam otimizar uma função de custo com objetivo de gerar o menor erro possível de acordo com a otimização do erro da função de custo. À medida que a rede geradora muda de comportamento, a rede discriminadora também muda seu comportamento. O objetivo da rede geradora é enganar a rede discriminadora (maximizar o erro de classificação final entre dados verdadeiros e dados fakes) e o objetivo do discriminador é detectar dados fakes (minimizar o erro de classificação final).

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

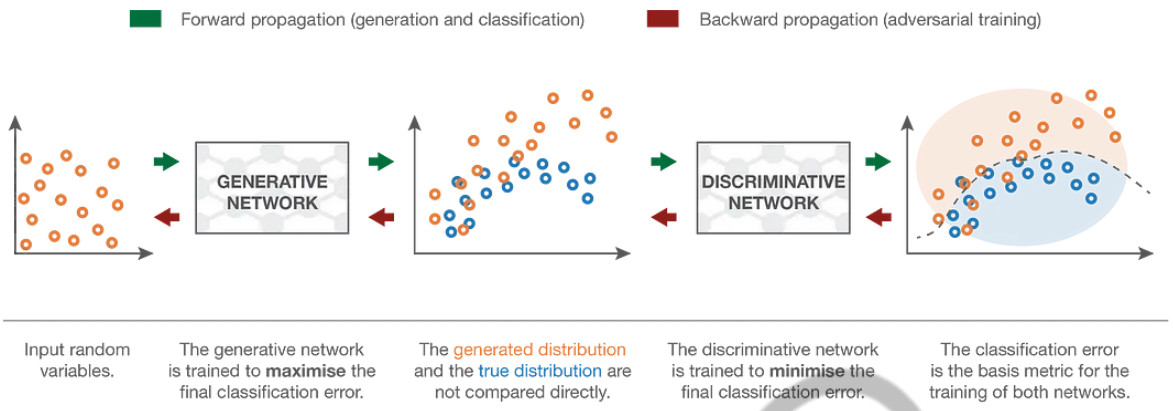


Figura 2 - Representação de Redes Adversariais Generativas
Fonte:https://towardsdatascience.com/understanding-generative-adversarial-networks-gans-cd6e4651a29 (2019)

É uma ideia genial, mas também muito perigosa por conta das **Deep Fakes** . Veja aqui um exemplo em que realizaram deep fake com o ex-presidente dos Estados Unidos, Barack Obama.

Observe a seguir uma figura que representa o comportamento dessas redes criando novas capas de álbuns da banda Pink Floyd.

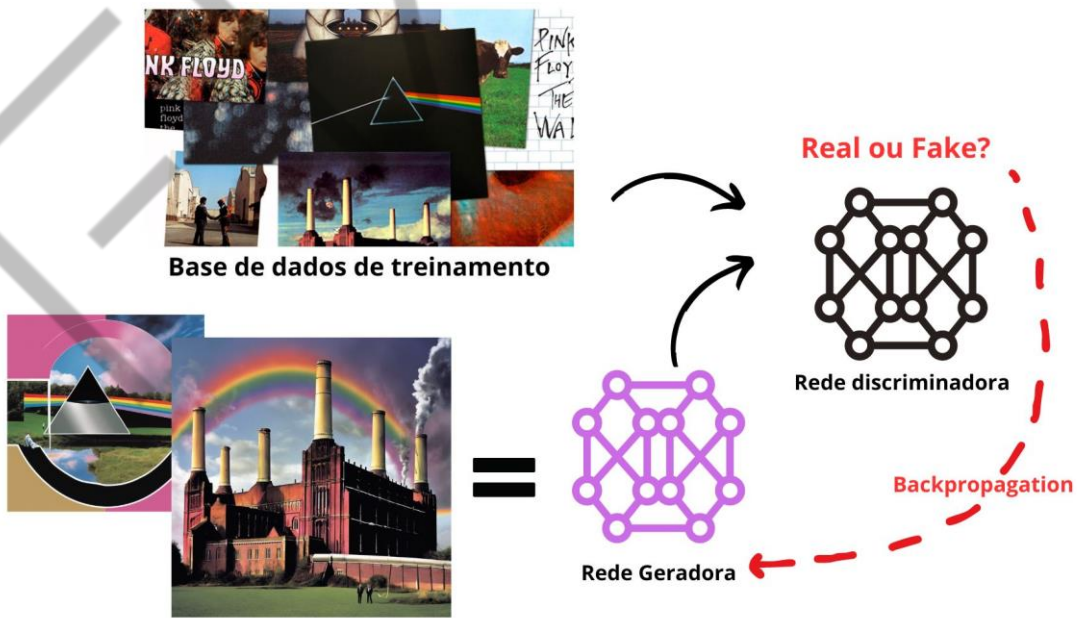


Figura 3 - Rede GANs
Fonte: elaborado pela autora (2024)

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

Entendendo a criação de vídeos com a IA generativa

Para a IA generativa criar vídeos, basicamente as redes **adversárias GANs** e **redes convolucionais** são utilizadas, porém, sendo realizada em um ritmo sincronizado. Para aprender a gerar vídeo, também entra nessa arquitetura as chamadas redes **LSTM (Long Short Term Memory)**. Essa rede LSTM tem o objetivo de **aprender e modelar dependências ao longo prazo.** Vamos entender agora como funciona sua arquitetura para compreender o sentido do uso das LSTM para gerar vídeos nas redes GANs com base em frames de vídeos.

LSTM é um tipo especial de rede neural recorrente, pois são capazes de aprender conexões de longo prazo. Dessa maneira, elas têm um incrível poder de predição e funcionam muito bem em uma variada gama de problemas que envolvem capturas de padrões de longo prazo em séries temporais, textos longos, gravações de áudios, vídeos e muito mais. O principal trabalho dessa rede neural se torna **entender como uma memória deve influenciar a outra**, ou seja, como a rede decide quais partes da memória de curto prazo devem ser lembradas e como a memória de longo prazo deve afetar o entendimento atual do texto.

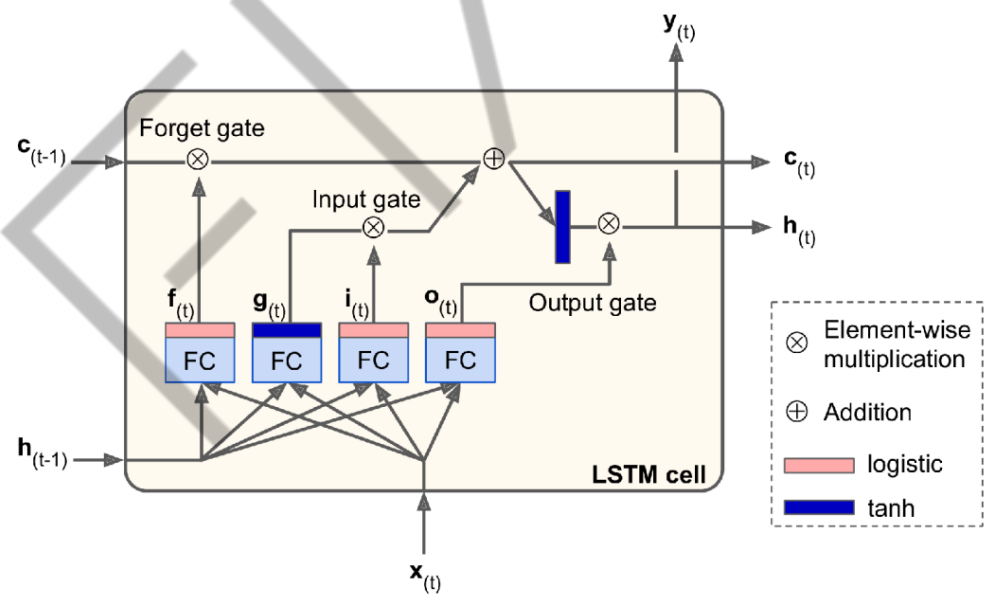


Figura 4 - Célula de uma rede neural recorrente LSTM
Fonte: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow [s.d.]

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

As células LSTM gerenciam dois vetores de estado e, por razões de desempenho, eles são mantidos separados por padrão: vetores **ht** e **ct** (o “c” se refere à célula). Podemos pensar que o **ht é um estado de curto** prazo e **ct de longo prazo**.

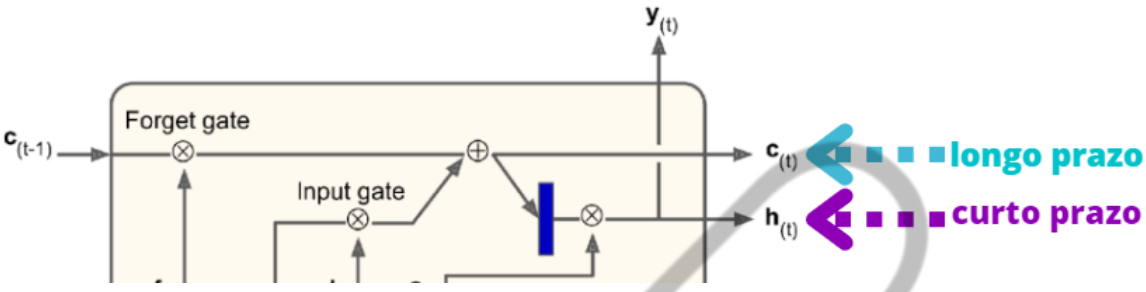


Figura 5 - vetores de estado de memórias LSTM
Fonte: elaborado pela autora (2024)

Como o estado do longo prazo **ct** atravessa a rede da esquerda para a direita, podemos observar um primeiro portão de entrada dos dados que seria o **forget gate**.

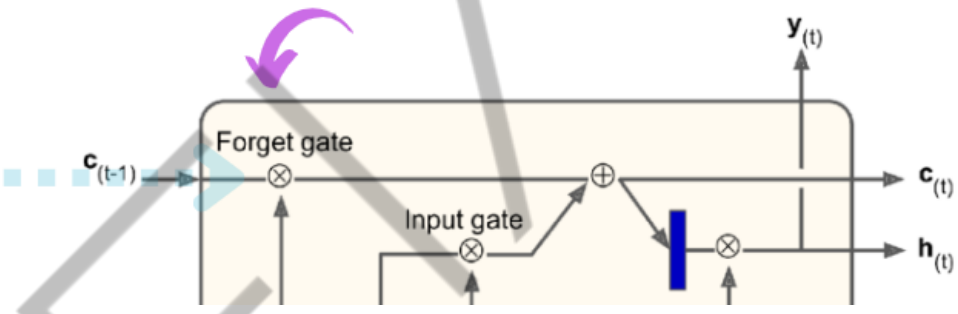


Figura 6 - forget gate LSTM
Fonte: elaborado pela autora (2024)

O forget gate é responsável por descartar algumas memórias e adicionar algumas novas memórias ao longo de adições de memórias que vem do **input gate**.

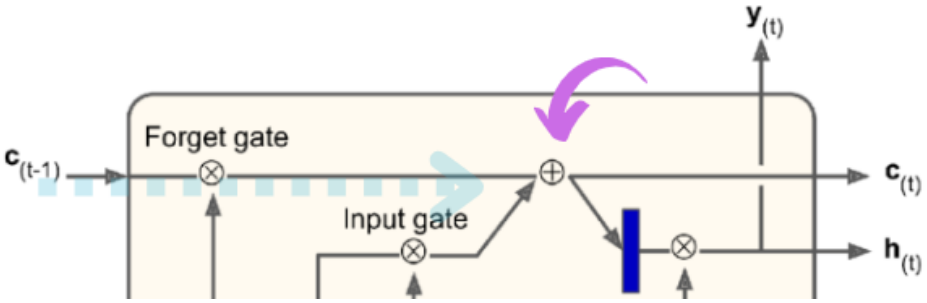


Figura 7 - input gate LSTM
Fonte: elaborado pela autora (2024)

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

Logo após a operação de adição, o estado de longo prazo é copiado e passado pela função de ativação tangente hiperbólica (tanh) e em seguida seu resultado é filtrado pelo **output gate**, produzindo o estado de curto prazo em ht.

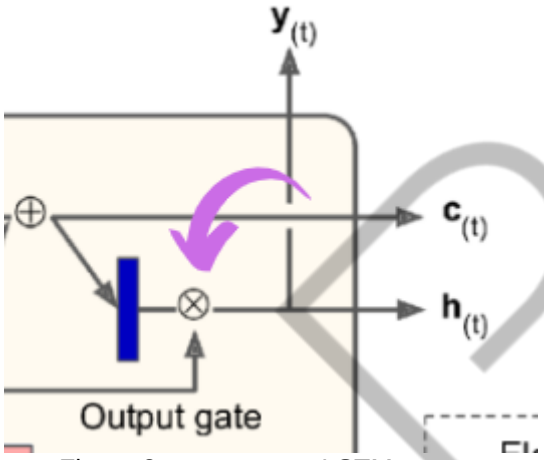


Figura 8 - output gate LSTM
Fonte: elaborado pela autora (2024)

Funcionamento das portas de memórias

O vetor de entrada atual **xt** e o estado de curto prazo anterior **h(t-1)** são fornecidos para quatro camadas diferentes conectadas. A camada que produz g(t) assume o papel de analisar as entradas atuais em x(t) e o estado anterior de curto prazo h(t-1). A saída dessa célula basicamente vai para y(t) e h(t).

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

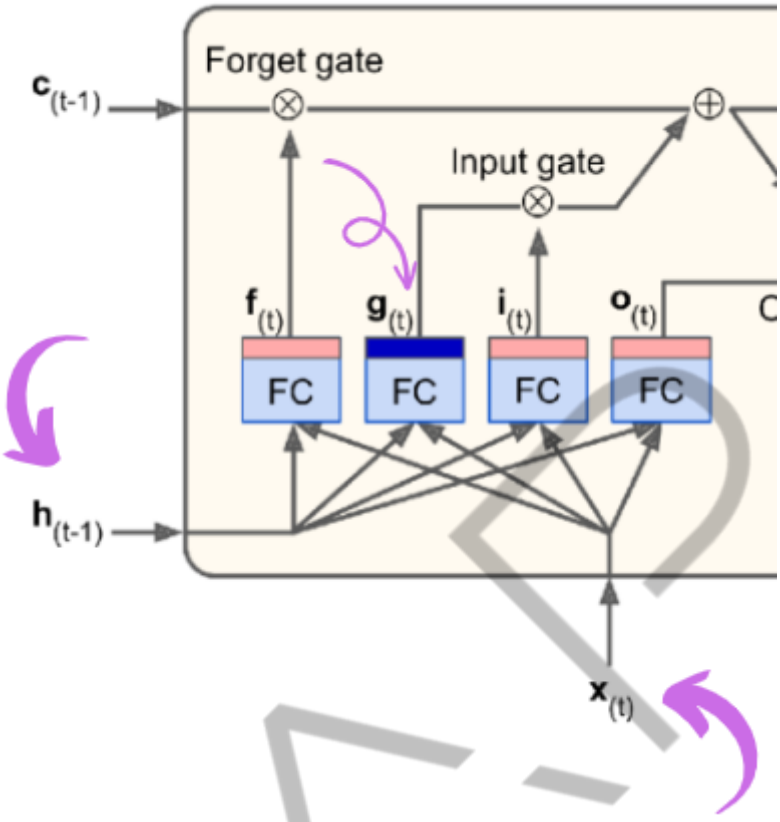
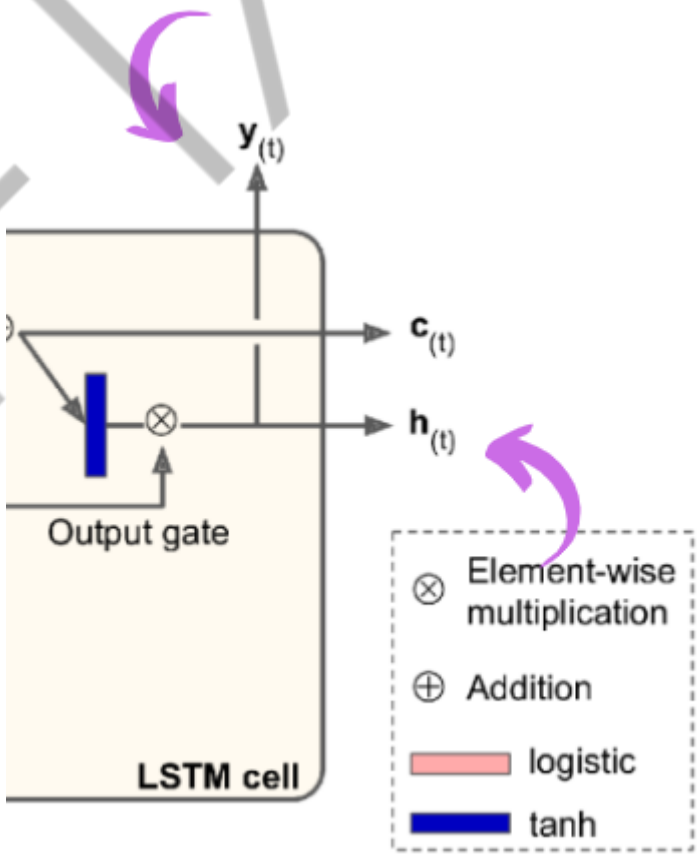


Figura 9 - Camada g(t) da LSTM
Fonte: elaborado pela autora (2024)



PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

Figura 10 - Camada de saída (t) e h(t) da LSTM
Fonte: elaborado pela autora (2024)

As outras três camadas (forget gate, input gate e output gate) são os chamados gates controllers. Esses portões utilizam funções de ativação logística que variam de 0 a 1.

- **Forget gate (ft):** decide quais partes do cell state continuam importantes (exclui as partes que devem ser apagadas);
- **Input gate (it):** decide quais informações da memória de curto prazo devem ser adicionadas ao cell state (célula de longa memória);
- **Output gate (ot):** decide quais partes do cell state são importantes no instante atual para gerar o output.

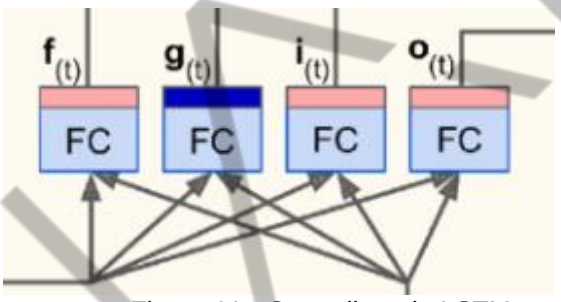


Figura 11 - Controllers da LSTM
Fonte: elaborado pela autora (2024)

Em resumo, podemos definir que a LSTM pode aprender a reconhecer uma entrada importante (realizada pelo input gate) e armazená-lo no estado de longo prazo (forget gate) e aprender a extraí-la sempre que necessário.

Cálculos de uma única instância da LSTM:

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

$$
\begin{aligned}
\mathbf{i}_{(t)} &= \sigma\left(\mathbf{W}_{xi}{}^{\mathsf{T}}\mathbf{x}_{(t)} + \mathbf{W}_{hi}{}^{\mathsf{T}}\mathbf{h}_{(t-1)} + \mathbf{b}_i\right) \\
\mathbf{f}_{(t)} &= \sigma\left(\mathbf{W}_{xf}{}^{\mathsf{T}}\mathbf{x}_{(t)} + \mathbf{W}_{hf}{}^{\mathsf{T}}\mathbf{h}_{(t-1)} + \mathbf{b}_f\right) \\
\mathbf{o}_{(t)} &= \sigma\left(\mathbf{W}_{xo}{}^{\mathsf{T}}\mathbf{x}_{(t)} + \mathbf{W}_{ho}{}^{\mathsf{T}}\mathbf{h}_{(t-1)} + \mathbf{b}_o\right) \\
\mathbf{g}_{(t)} &= \tanh\left(\mathbf{W}_{xg}{}^{\mathsf{T}}\mathbf{x}_{(t)} + \mathbf{W}_{hg}{}^{\mathsf{T}}\mathbf{h}_{(t-1)} + \mathbf{b}_g\right) \\
\mathbf{c}_{(t)} &= \mathbf{f}_{(t)} \otimes \mathbf{c}_{(t-1)} + \mathbf{i}_{(t)} \otimes \mathbf{g}_{(t)} \\
\mathbf{y}_{(t)} &= \mathbf{h}_{(t)} = \mathbf{o}_{(t)} \otimes \tanh\left(\mathbf{c}_{(t)}\right)
\end{aligned}
$$

Figura 12 - Equação de célula LSTM
Fonte: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow [s.d.]

No caso da RNN, o vanishing gradient ocorre por causa da função de ativação. Como na célula de memória não é aplicada nenhuma função de ativação (as funções são aplicadas só nas gates), não temos esse tipo de problema.

Na rede geradora, a LSTM pode ser utilizada para processar o vetor de ruído aleatório e gerar uma sequência de frames de vídeo, capturando as dependências temporais entre os frames e criando vídeos mais fluidos e realistas. E na rede discriminadora, pode ser utilizada para analisar a sequência de frames de vídeo e determinar se o vídeo é real ou falso, levando em consideração a coerência temporal e a fluidez do movimento.

Entendendo a criação de códigos com a IA generativa

Se você já é uma pessoa que trabalha com programação no seu dia a dia, provavelmente já utilizou alguma IA generativa para ajustar algum código seu desenvolvido em alguma linguagem de programação. Eu sou cientista de dados e utilizo muito no meu dia a dia as IA generativas como o Google Gemini, Chat GPT, Copilot e entre outras IA generativas para ajustar o código. Vamos analisar um exemplo:

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

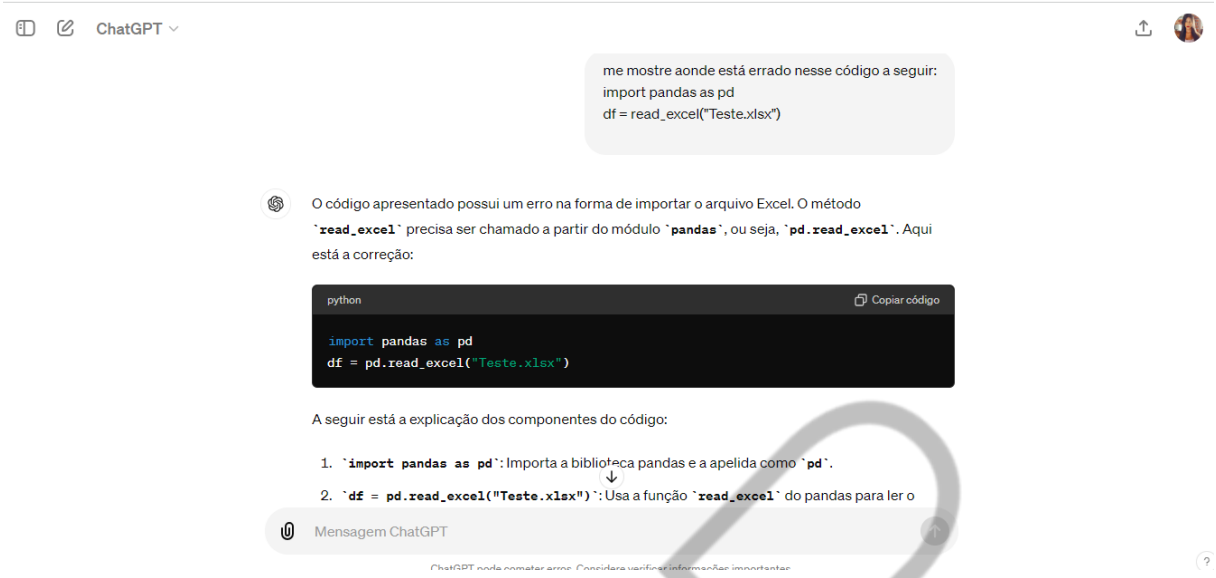


Figura 13 - Testando a geração de código no Chat GPT
Fonte: elaborado pela autora (2024)

Fiz um teste bem simples para mostrar que ao pedir para a IA corrigir o código, logo já foi identificado que estava faltando o apelido da biblioteca do pandas, o “pd”, antes de utilizar a função de read_excel. Sensacional né? Se você pedir a criação de códigos complexos, a rede também consegue gerar. Veja o exemplo a seguir.

Meu prompt foi o seguinte: **“crie uma deep learning convolucional para classificar imagens”**, observe o incrível output:

Para criar um modelo de deep learning convolucional (CNN) para classificar imagens, você pode usar a biblioteca Keras com o TensorFlow como backend. Aqui está um exemplo de como você pode construir e treinar uma CNN para classificação de imagens:

1. **Importar as bibliotecas necessárias:**

python

Copiar código

```
import tensorflow as tf

from tensorflow.keras import layers, models
```

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

1. **Preparar os dados:**

Supondo que você tenha um conjunto de dados de imagens organizadas em diretórios para treinamento e validação, você pode usar `ImageDataGenerator` para carregar e pré-processar essas imagens.

```
# Configurações do gerador de dados
train_datagen = ImageDataGenerator(rescale=1./255,
                                   shear_range=0.2,
                                   zoom_range=0.2,
                                   horizontal_flip=True)

val_datagen = ImageDataGenerator(rescale=1./255)

# Diretórios de treinamento e validação
train_dir = 'path_to_train_directory'
val_dir = 'path_to_validation_directory'

# Carregar dados de treinamento e validação
```

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

```
train_generator = train_datagen.flow_from_directory(train_dir,

target_size=(150, 150),

batch_size=32,

class_mode='binary')  # 'binary' para classificação binária ou
'categorical' para classificação multiclasse

val_generator = val_datagen.flow_from_directory(val_dir,

target_size=(150, 150),
                                                batch_size=32,

class_mode='binary')
```

1. **Construir o modelo CNN:**

```
python

model = models.Sequential()

# Primeira camada de convolução
```

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

```
model.add(layers.Conv2D(32,    (3,    3),    activation='relu',
input_shape=(150, 150, 3)))

model.add(layers.MaxPooling2D((2, 2)))



# Segunda camada de convolução

model.add(layers.Conv2D(64, (3, 3), activation='relu'))

model.add(layers.MaxPooling2D((2, 2)))



# Terceira camada de convolução

model.add(layers.Conv2D(128, (3, 3), activation='relu'))

model.add(layers.MaxPooling2D((2, 2)))



# Quarta camada de convolução

model.add(layers.Conv2D(128, (3, 3), activation='relu'))

model.add(layers.MaxPooling2D((2, 2)))



# Camada de flattening

model.add(layers.Flatten())



# Camada densa com dropout
```

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

```
model.add(layers.Dense(512, activation='relu'))

model.add(layers.Dropout(0.5))



# Camada de saída

model.add(layers.Dense(1,   activation='sigmoid'))      #   Use
'softmax' para classificação multiclasse



# Compilar o modelo

model.compile(optimizer='adam',

              loss='binary_crossentropy',            #      Use
'categorical_crossentropy' para classificação multiclasse

              metrics=['accuracy'])
```

1. **Treinar o modelo:**

```
python

history = model.fit(train_generator,

                    steps_per_epoch=100,   # Número de batches
por epoch

                    epochs=30,  # Número de epochs

                    validation_data=val_generator,
```

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

```
                    validation_steps=50)  # Número de batches de
validação por epoch
```

1. **Avaliar o modelo:**

python

```
# Avaliar o modelo nos dados de validação

val_loss, val_acc = model.evaluate(val_generator, steps=50)

print(f'Loss: {val_loss}')

print(f'Accuracy: {val_acc}')
```

1. **Salvar o modelo:**

python

```
model.save('meu_modelo_cnn.h5')
```

Este é um exemplo básico de um modelo CNN para classificação de imagens. Dependendo da complexidade do seu conjunto de dados e do problema específico, você pode precisar ajustar a arquitetura do modelo, os parâmetros de treinamento, e outros hiperparâmetros.

Incrível, né? Então as IA generativas conseguem nos auxiliar tanto em códigos simples, quanto em códigos complexos. Mas a engenharia de prompt é muito importante nesse contexto para facilitar a forma como perguntamos para o modelo a geração de conteúdo. A ideia é ser o mais simples possível, como se estivéssemos explicando algo para uma pessoa. Existem técnicas de engenharia de prompt que podem te auxiliar nas perguntas a serem realizadas para o prompt, e este artigo pode te auxiliar com as principais técnicas.

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

Mas como acontece essa mágica? Além de toda arquitetura já discutida nesta aula, tal como as redes neurais recorrentes e as redes convolucionais, basicamente nesse cenário de códigos, temos fortemente a ação da rede **Transformers** contribuindo para a geração do conteúdo preciso e correto (assim como também é utilizado para criação de texto). Vamos entender como funciona um pouco mais a fundo essa arquitetura.

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

Entendendo o modelo transformers

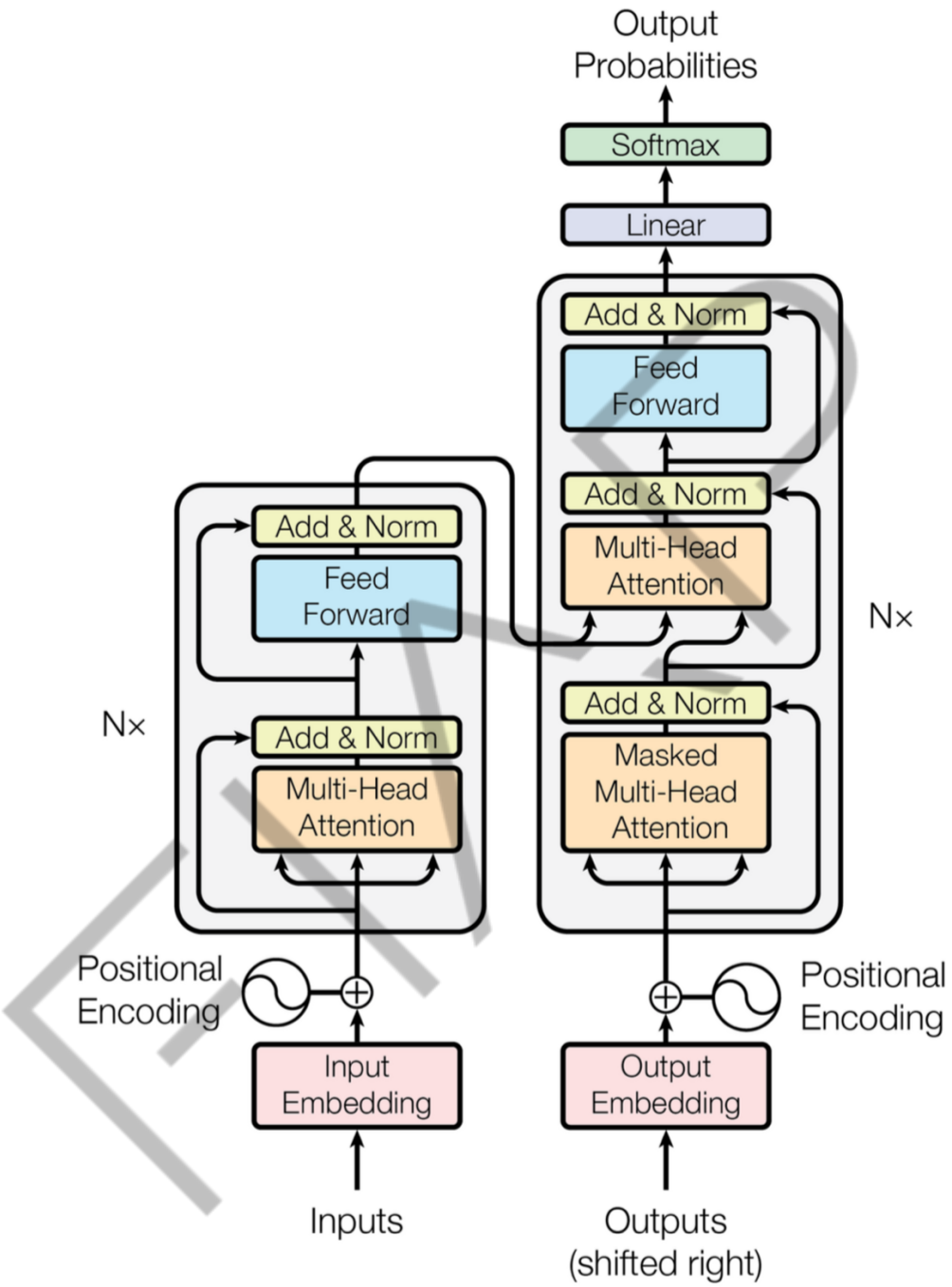


Figure 1: The Transformer - model architecture.

Figura 14 - Arquitetura Transformers
Fonte: https://erika-gl-alves.medium.com/um-breve-estudo-dos-transformers-6abbf1b77512 (2022)

Como já mencionamos na aula de introdução, os modelos transformers utilizam o mecanismo de **atenção** (Attention Mechanism), apresentado pelo Google em 2017. Por trás da magia desse modelo de atenção, inicialmente temos os componentes de

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

codificação e decodificação trabalhando entre si para buscar as relações entre as palavras. O artigo original empilha seis camadas de cada um deles, mas esse valor pode ser alterado caso seja necessário.

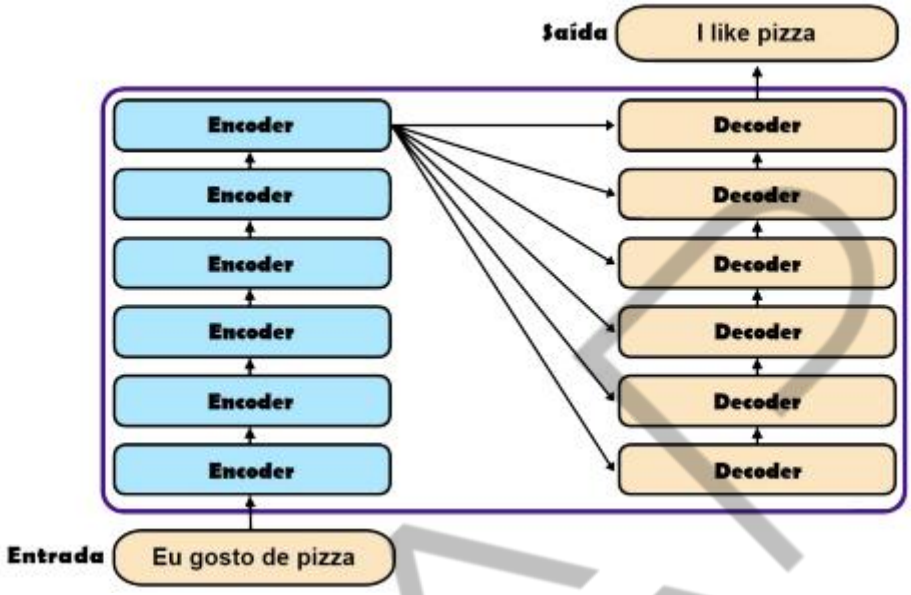


Figura 15 - Camada de encoders e decoders empilhadas
Fonte: Inteligência Artificial e ChatGPT: Da revolução dos modelos de IA generativa à Engenharia de Prompt (2022)

Vamos entender por partes:

Encoder (codificação): essa parte da arquitetura da rede analisa cada palavra de entrada encontrando suas relações semânticas em conjunto com uma camada de **embeddings** (responsável por criar a representação vetorial de semântica das palavras). Depois de encontrar os embeddings correspondentes nessa etapa, esses tokens são atributos a uma camada da rede de "**Multi-Head Attention"** (ajuda o encoder a examinar as outras palavras da frase enquanto codifica uma palavra específica) e, em seguida, a saída vai para uma camada de **"Feed Foward"** para alimentar a camada de **decoder** que também possui uma camada **"Feed Foward"**.

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

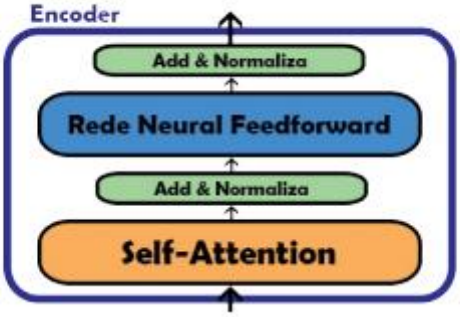


Figura 16 - Camada de encoder
Fonte: Inteligência Artificial e ChatGPT: Da revolução dos modelos de IA generativa à Engenharia de Prompt (2022)

Decoder (decodificação): a camada de decoder possui as mesmas camadas da encoder, porém com uma diferença: a **camada intermediária self-attention,** que recebe diretamente o resultado da camada de encoders, **com o objetivo de foco de atenção nas partes mais importantes da frase de entrada (é o método incorpora o "entendimento" de outras palavras relevantes na palavra que estamos processando no momento)**. Essa camada self-attention só consegue focar nas palavras anteriores a etapa atual, por isso também é chamada de **"masked"** (camuflada). Essa etapa utiliza um **embedding** para capturar qual é a palavra que faz mais sentido ser retornada no prompt de geração de conteúdo com base no seu maior potencial de probabilidade para gerar o output.

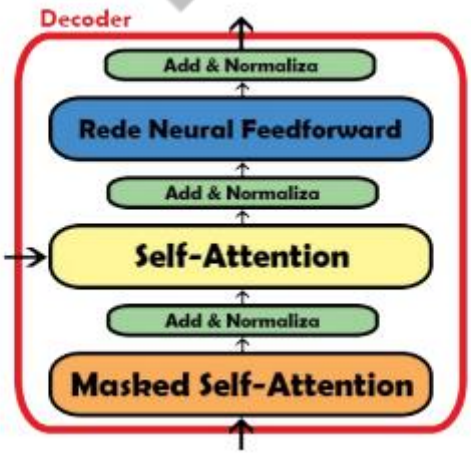


Figura 17- Camada de decoder
Fonte: Inteligência Artificial e ChatGPT: Da revolução dos modelos de IA generativa à Engenharia de Prompt

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

Depois na camada de saída final das palavras selecionadas pelo decoder, será realizada uma **transformação linear aos vetores de saída de todas as subcamadas de atenção do decoder** para gerar uma saída de um vocabulário. Essa transformação visa **combinar as informações** de diferentes subcamadas, capturando uma representação mais abrangente do significado da entrada e do contexto da geração. Após a transformação linear, a **função softmax** entra em cena. Ela aplica uma **função de normalização** aos vetores de saída, garantindo que os valores fiquem entre **0 e 1** e que a soma de todos os valores seja **1**. Como a softmax é uma função de ativação que seleciona a maior probabilidade de pertencimento de uma classe, a softmax vai selecionar qual é a palavra que tem mais probabilidade de ser a mais “provável” como resposta do modelo.

Funcionamento da Multi-Head Self Attention

Você percebeu que na camada de encoder e decoder falamos de um processo de atenção de aprendizagem nas palavras principais? Pois é, esse é o processo de **Multi-Head Self Attention.** Em vez de focar apenas em uma palavra por vez, a Multi-Head Self Attention divide a atenção em múltiplas **"cabeças".** Cada "cabeça" se concentra em um aspecto diferente da relação entre as palavras, como proximidade, similaridade gramatical ou significado semântico. As informações de todas as "cabeças" são então **combinadas** para criar uma representação mais rica e completa do significado da frase. Observe a imagem a seguir, é como se cada cor que está sendo mapeada por cada palavra, representasse uma “cabeça” diferente com diferentes contextos.

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

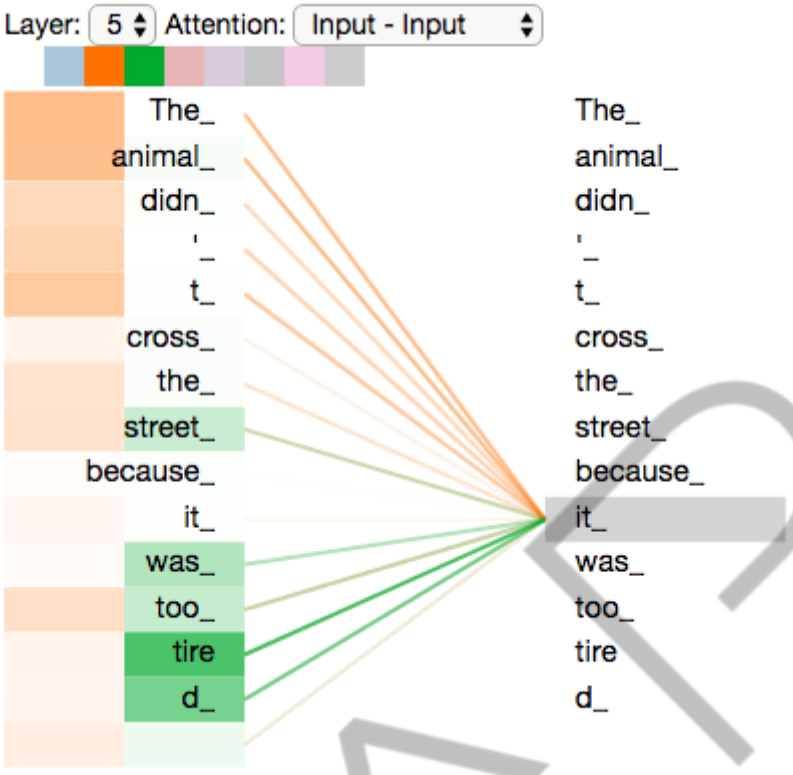


Figura 18 - Multi-Head Self Attention focando em palavras e contextos distintos
Fonte: https://brains.dev/2023/os-transformers-ilustrados/ (2022)

A entrada para a Multi-Head Self Attention é um conjunto de **vetores de palavras**, onde cada palavra é representada por um vetor de números. Cada vetor de palavra é então projetado em **três espaços distintos** usando **matrizes de projeção** aprendidas durante o treinamento. Esses espaços são chamados de **Query (Q), Key (K) e Value (V)**. Para cada palavra, a Multi-Head Self Attention calcula uma **pontuação de atenção** para cada outra palavra na frase.

Essa pontuação indica a **importância** da outra palavra para entender o significado da palavra em questão. A pontuação de atenção é calculada usando o **produto escalar** dos vetores **Query (Q)** e **Key (K)** das duas palavras. Essa operação mede a **similaridade** entre as duas palavras. As pontuações de atenção são então **normalizadas** para garantir que todas estejam na mesma escala e que a contribuição de cada palavra seja ponderada de forma justa.

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

O processo de cálculo da pontuação de atenção é repetido **múltiplas vezes**, usando diferentes projeções de **Query (Q), Key (K) e Value (V)** para cada "cabeça". Isso permite que o modelo capture diferentes tipos de relações entre as palavras. As pontuações de atenção de todas as "cabeças" são então **combinadas** em uma única pontuação de atenção final para cada palavra. O vetor **Value (V)** de cada palavra é multiplicado pela sua pontuação de atenção final, criando um novo vetor que representa o **contexto** da palavra. Os vetores de contexto de todas as palavras são então **somados**, resultando em um único vetor de contexto que representa o significado geral da frase. O vetor de contexto final é a saída da Multi-Head Self Attention. Ele representa uma **visão rica e complexa do significado da frase**, capturando as relações entre as palavras e o contexto geral.

Os Transformers são treinados usando grandes quantidades de dados de texto, como livros, artigos e websites. Durante o treinamento, a rede ajusta seus parâmetros para minimizar a diferença entre a saída gerada e a saída desejada (tradução correta, resposta precisa, etc.). Por isso que são tão precisos para criar códigos complexos e com precisão. Embora o modelo tradicional de transformers trabalhe com encoder e decoder, essas duas camadas nem sempre trabalham em conjunto em outros tipos de modelos transformadores. Os modelos bidirecionais, tal como o BERT, ALBERT, DistilBERT, ELECTRA e RoBERTa, apenas utilizam encoder para compreender o contexto pela direita e pela esquerda (daí o nome bidirecional). O modelo GPT e GPT2 já trabalha especificamente com os decoders, sendo ótimos modelos para geração de texto, sendo um modelo unidirecional e analisando o contexto apenas pela esquerda.

Você quer conhecer algumas IA generativas que realizam todo esse aprendizado? Que tal dar uma olhadinha neste artigo?

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nessa aula, você aprendeu como podemos utilizar a IA generativa para gerar diversos tipos de conteúdos e como funcionam as técnicas por trás dos modelos generativos.

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

REFERÊNCIAS

ALVES, E. Um breve estudo dos Transformers. **Medium**. 2022. Disponível em: https://erika-gl-alves.medium.com/um-breve-estudo-dos-transformers-6abbf1b77512. Acesso em: 25 jun. 2024.

CAPÍTULO 54 – Introdução às redes Adversárias Generativas (GANs – Generative Adversarial Networks). **Deep Learning Book**. 2022. Disponível em: https://www.deeplearningbook.com.br/introducao-as-redes-adversarias-generativas-gans-generative-adversarial-networks/. Acesso em: 25 jun. 2024.

CAPÍTULO 85 – Transformadores - o estado da arte em processamento natural. **Deep Learning Book.** 2022. Disponível em: https://www.deeplearningbook.com.br/transformadores-o-estado-da-arte-em-processamento-de-linguagem-natural/. Acesso em: 25 jun. 2024.

CARRARO, F. **Inteligência artificial e chatGPT: da revolução dos modelos de IA generativa à engenharia de prompt**. São Paulo: Casa do código, 2023.

GÉRON, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. 2ª ed. Califórnia: O'Reilly Media, Inc., 2019.

LÓPES, A. Os Transformers Ilustrados. **Brains.** 2023. Disponível em: https://brains.dev/2023/os-transformers-ilustrados/ Acesso em: 25 jun. 2024.

ROCCA, J. Understanding Generative Adversarial Networks (GANs). **Medium**. 2019. Disponível em: https://towardsdatascience.com/understanding-generative-adversarial-networks-gans-cd6e4651a29. Acesso em: 25 jun. 2024.

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

PALAVRAS-CHAVE

IA generativa. GPT. LLMs. GANs. Deep Learning. LSTM.

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

POS TECH

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com